

# SISTEMAS DISTRIBUÍDOS

# DINÂMICA DE LABORATÓRIO —

# INTEGRALAB

*Interoperabilidade REST e gRPC orientada por contratos*

## GUIA DO GRUPO CLIENTE

Você desenvolverá 1 Cliente REST e 1 Cliente gRPC.

**Cenário: ConectaShop — marketplace distribuído**

*Tecnologia de implementação: livre escolha da equipe*

## 1. Sua missão nesta dinâmica

**Contexto.** A ConectaShop está decompondo seu marketplace em serviços independentes. Diferentes equipes desenvolverão componentes em tecnologias potencialmente distintas e só descobrirão com quem irão integrar depois da implementação.

**Seu papel:** CLIENTE. Sua equipe é responsável por construir consumidores independentes dos dois contratos. Seu código deve ser capaz de apontar para Servidores desconhecidos e executar automaticamente os testes definidos neste documento.

### Regra fundamental

O contrato é a única fonte de verdade. Depois do sorteio, não será permitido alterar paths, nomes de campos, tipos, códigos de status, package/service/métodos gRPC ou regras de cálculo para adaptar a solução a uma equipe específica.

## 2. Como a dinâmica funciona

1. O professor divide a turma entre grupos CLIENTE e grupos SERVIDOR.
2. Todos recebem os mesmos contratos REST e gRPC, mas cada grupo implementa apenas o seu lado.
3. Cada equipe escolhe livremente linguagem, framework, bibliotecas e organização do projeto.
4. Ao final da fase de implementação, o professor sorteia as integrações.
5. Cada grupo Cliente executa seus testes contra dois Servidores REST e dois Servidores gRPC.
6. Cada grupo Servidor é testado por dois Clientes REST e dois Clientes gRPC.
7. Toda tentativa deve gerar logs. Integrações que falharem também devem ser registradas e analisadas.
8. A entrega final inclui código, instruções de execução, logs e relatório técnico das integrações.

## 3. Regras que valem para todos

- A tecnologia é livre, mas o contrato é obrigatório.
- Não compartilhe código-fonte entre Cliente e Servidor para adaptar uma integração específica.
- Após o sorteio, as equipes podem trocar somente informações operacionais: IP/host, porta, código da equipe e estado do serviço.
- Toda chamada deve permitir correlação entre Cliente e Servidor por identificadores de equipe e requisição.
- Falha de integração não deve ser escondida: deve ser registrada, classificada e explicada.

## 4. O que sua equipe deve produzir

| Artefato           | Obrigatório | O que deve conter                                                                 |
|--------------------|-------------|-----------------------------------------------------------------------------------|
| Cliente REST       | Sim         | Programa que execute R1–R5 contra REST_BASE_URL configurável.                     |
| Cliente gRPC       | Sim         | Programa que execute G1–G5 contra GRPC_TARGET configurável.                       |
| README             | Sim         | Pré-requisitos, instalação, configuração e comandos de execução.                  |
| Logs de integração | Sim         | Logs de todas as tentativas com destino, request ID, status, duração e resultado. |
| Relatório final    | Sim         | Matriz das 4 integrações, evidências, falhas encontradas e análise técnica.       |

| Artefato     | Obrigatório | O que deve conter                                                     |
|--------------|-------------|-----------------------------------------------------------------------|
| Código-fonte | Sim         | Projeto completo, sem depender de código de Servidor de outra equipe. |

## 5. Requisitos específicos do Cliente

- Aceitar REST\_BASE\_URL de forma configurável, sem recompilar.
- Aceitar GRPC\_TARGET no formato host:porta de forma configurável, sem recompilar.
- Enviar X-Client-Team e X-Request-ID em todas as chamadas REST.
- Enviar metadata x-client-team em todas as chamadas gRPC.
- Gerar um request\_id único para cada chamada gRPC.
- Exibir claramente protocolo, operação, destino, request, status/resposta, duração aproximada e resultado PASS/FAIL.
- Não assumir linguagem, framework, sistema operacional, banco de dados ou estrutura interna do Servidor.
- Executar R1–R5 e G1–G5 contra os Servidores sorteados.

### Configuração obrigatória

Seu programa deve poder trocar de Servidor rapidamente. Exemplos válidos: variáveis de ambiente, argumentos de linha de comando ou arquivo de configuração. Endereços não podem ficar hardcoded.

## Contrato REST — Catalog & Quote API

Este é o contrato que deve orientar toda a implementação REST. **Não invente campos, paths, headers ou regras adicionais.**

### Propriedades da API

| Propriedade    | Valor                                            |
|----------------|--------------------------------------------------|
| Nome lógico    | Catalog & Quote API                              |
| Versão         | v1                                               |
| Base path      | /api/v1                                          |
| Formato        | application/json; charset=utf-8                  |
| Porta sugerida | 8080 (o Servidor pode informar outra no sorteio) |

### Headers

| Header        | Obrigatório        | Descrição                                                                                                       |
|---------------|--------------------|-----------------------------------------------------------------------------------------------------------------|
| X-Client-Team | Sim                | Código da equipe Cliente, por exemplo C03.                                                                      |
| X-Request-ID  | Sim                | Identificador único da chamada. Deve aparecer nos logs de ambos os lados e ser devolvido no header da resposta. |
| Content-Type  | Quando houver body | application/json                                                                                                |

**Atenção**

Se X-Client-Team ou X-Request-ID não estiver presente, o Servidor deve responder HTTP 400 com code MISSING\_REQUIRED\_HEADER. Em respostas válidas, X-Request-ID deve ser devolvido no header.

**Catálogo fixo**

| SKU    | Nome             | unitPriceCents | available |
|--------|------------------|----------------|-----------|
| KB-100 | Teclado Mecânico | 25990          | true      |
| MS-200 | Mouse Sem Fio    | 12990          | true      |
| HD-300 | Headset USB      | 19990          | true      |
| MN-400 | Monitor 27       | 119990         | true      |

**Valores monetários:** sempre em centavos inteiros. Ex.: 25990 = R\$ 259,90.

**Endpoint 1 — Consultar produto**

```
GET /api/v1/products/{sku}
```

```
GET /api/v1/products/KB-100
X-Client-Team: C01
X-Request-ID: req-001
```

Resposta de sucesso — HTTP 200:

```
{
  "sku": "KB-100",
  "name": "Teclado Mecânico",
  "unitPriceCents": 25990,
  "available": true
}
```

Produto inexistente — HTTP 404:

```
{
  "code": "PRODUCT_NOT_FOUND",
  "message": "Product not found",
  "requestId": "req-002"
}
```

**Endpoint 2 — Calcular cotação**

```
POST /api/v1/quotes
```

Body de exemplo:

```
{
  "items": [
    { "sku": "KB-100", "quantity": 2 },
    { "sku": "MS-200", "quantity": 1 }
  ]
}
```

#### Validações obrigatórias:

- items deve conter de 1 a 5 itens distintos.
- quantity deve ser inteiro entre 1 e 10.
- todo SKU deve existir no catálogo fixo.
- SKUs duplicados no mesmo request são inválidos; o Cliente deve consolidar quantidades antes de enviar.

#### Regra de cálculo:

9.  $\text{subtotalCents} = \text{soma de unitPriceCents} \times \text{quantity}$ .
10. Desconto:  $\text{subtotal} < 50000 \rightarrow 0\%$ ;  $50000-99999 \rightarrow 5\%$ ;  $\text{subtotal} \geq 100000 \rightarrow 10\%$ .
11.  $\text{discountCents} = \text{parte inteira de } (\text{subtotalCents} \times \text{discountPercent} / 100)$ .
12.  $\text{totalCents} = \text{subtotalCents} - \text{discountCents}$ .

| Item          | Cálculo           |
|---------------|-------------------|
| 2 × KB-100    | 2 × 25990 = 51980 |
| 1 × MS-200    | 1 × 12990 = 12990 |
| Subtotal      | 64970             |
| Desconto      | 5%                |
| discountCents | 3248              |
| totalCents    | 61722             |

#### Resposta oficial para o exemplo:

```
{
  "requestId": "req-003",
  "subtotalCents": 64970,
  "discountPercent": 5,
  "discountCents": 3248,
  "totalCents": 61722
}
```

## Erros REST

| Situação                                      | HTTP | code                      |
|-----------------------------------------------|------|---------------------------|
| Header obrigatório ausente                    | 400  | MISSING_REQUIRED_HEADER   |
| JSON inválido ou campo obrigatório ausente    | 400  | INVALID_REQUEST           |
| Produto consultado não existe                 | 404  | PRODUCT_NOT_FOUND         |
| SKU inválido na cotação                       | 422  | INVALID_PRODUCT           |
| Quantidade fora do intervalo ou SKU duplicado | 422  | INVALID_QUANTITY_OR_ITEMS |

| Situação                    | HTTP | code           |
|-----------------------------|------|----------------|
| Erro inesperado do servidor | 500  | INTERNAL_ERROR |

Formato padrão de erro:

```
{
  "code": "INVALID_PRODUCT",
  "message": "Human readable description",
  "requestId": "req-004"
}
```

## Testes REST obrigatórios

| Teste | Operação                        | Resultado esperado                                                         |
|-------|---------------------------------|----------------------------------------------------------------------------|
| R1    | GET KB-100                      | HTTP 200; unitPriceCents=25990                                             |
| R2    | GET XX-999                      | HTTP 404;<br>code=PRODUCT_NOT_FOUND                                        |
| R3    | POST quote: 2×KB-100 + 1×MS-200 | HTTP 200; subtotal=64970;<br>desconto=5; total=61722                       |
| R4    | POST quote: 1×MN-400            | HTTP 200; subtotal=119990;<br>desconto=10; discount=11999;<br>total=107991 |
| R5    | POST quote com SKU inexistente  | HTTP 422; code=INVALID_PRODUCT                                             |

### O que seu Cliente REST deve provar

Ele deve construir requests HTTP e interpretar responses apenas com base neste contrato, sem SDK ou código do grupo Servidor. Deve conseguir trocar REST\_BASE\_URL e repetir os mesmos testes contra qualquer Servidor sorteado.

## Contrato gRPC — ShippingService

Todos os grupos devem utilizar exatamente o arquivo **shipping.proto** abaixo. Não renomeie package, service, métodos, campos, enums nem números de campo.

```

syntax = "proto3";

package shipping.v1;

enum ShippingZone {
  SHIPPING_ZONE_UNSPECIFIED = 0;
  LOCAL = 1;
  REGIONAL = 2;
  NATIONAL = 3;
}

enum ShippingMode {
  SHIPPING_MODE_UNSPECIFIED = 0;
  STANDARD = 1;
  EXPRESS = 2;
}

service ShippingService {
  rpc CalculateShipping(ShippingRequest) returns (ShippingResponse);
  rpc Health(HealthRequest) returns (HealthResponse);
}

message ShippingRequest {
  string request_id = 1;
  int32 weight_grams = 2;
  ShippingZone zone = 3;
  ShippingMode mode = 4;
}

message ShippingResponse {
  string request_id = 1;
  int64 price_cents = 2;
  int32 estimated_days = 3;
  string server_team = 4;
}

message HealthRequest {}

message HealthResponse {
  string status = 1;
  string server_team = 2;
}

```

**Porta sugerida:** 50051. O Servidor pode utilizar outra, desde que informe host:porta no sorteio.

## Metadata obrigatória

| Chave         | Obrigatória | Descrição                                  |
|---------------|-------------|--------------------------------------------|
| x-client-team | Sim         | Código da equipe Cliente, por exemplo C01. |

Se x-client-team estiver ausente, a RPC deve terminar com status `INVALID_ARGUMENT` e descrição `MISSING_CLIENT_TEAM`.

## Regras de CalculateShipping

- `request_id` é obrigatório e não pode ser vazio.
- `weight_grams` deve estar entre 1 e 30000.
- `zone` não pode ser `SHIPPING_ZONE_UNSPECIFIED`.
- `mode` não pode ser `SHIPPING_MODE_UNSPECIFIED`.

- A resposta deve repetir request\_id exatamente como recebido.
- server\_team deve conter o código do grupo Servidor, por exemplo S02.

## Tarifa base

| Zona     | STANDARD | EXPRESS |
|----------|----------|---------|
| LOCAL    | 1000     | 1600    |
| REGIONAL | 1800     | 2800    |
| NATIONAL | 3000     | 4500    |

## Adicional por quilograma iniciado

| Modo     | Adicional                    |
|----------|------------------------------|
| STANDARD | 400 centavos por kg iniciado |
| EXPRESS  | 600 centavos por kg iniciado |

```
quilogramas_cobrados = teto(weight_grams / 1000)
price_cents = tarifa_base + quilogramas_cobrados × adicional
```

## Prazo estimado

| Zona     | STANDARD | EXPRESS |
|----------|----------|---------|
| LOCAL    | 2 dias   | 1 dia   |
| REGIONAL | 4 dias   | 2 dias  |
| NATIONAL | 7 dias   | 3 dias  |

## Health RPC

Health permite verificar se o Cliente alcançou o Servidor correto antes dos testes de negócio. Resposta esperada:

```
HealthResponse {
  status: "SERVING"
  server_team: "S02"
}
```

## Erros gRPC

| Situação                       | Status gRPC      | Descrição sugerida  |
|--------------------------------|------------------|---------------------|
| x-client-team ausente          | INVALID_ARGUMENT | MISSING_CLIENT_TEAM |
| request_id vazio               | INVALID_ARGUMENT | MISSING_REQUEST_ID  |
| weight_grams fora do intervalo | INVALID_ARGUMENT | INVALID_WEIGHT      |
| zone UNSPECIFIED               | INVALID_ARGUMENT | INVALID_ZONE        |

| Situação         | Status gRPC      | Descrição sugerida |
|------------------|------------------|--------------------|
| mode UNSPECIFIED | INVALID_ARGUMENT | INVALID_MODE       |
| Erro inesperado  | INTERNAL         | INTERNAL_ERROR     |

## Testes gRPC obrigatórios

| Teste | Entrada                   | Resultado esperado                                |
|-------|---------------------------|---------------------------------------------------|
| G1    | Health()                  | status=SERVING; server_team identifica o servidor |
| G2    | 1500g, LOCAL, STANDARD    | price_cents=1800; estimated_days=2                |
| G3    | 2500g, REGIONAL, EXPRESS  | price_cents=4600; estimated_days=2                |
| G4    | 1000g, NATIONAL, STANDARD | price_cents=3400; estimated_days=7                |
| G5    | weight_grams=0            | Status INVALID_ARGUMENT;<br>INVALID_WEIGHT        |

### O que seu Cliente gRPC deve provar

Sua equipe deve gerar os próprios stubs a partir deste .proto, configurar GRPC\_TARGET e executar G1–G5 contra qualquer Servidor sorteado. Não receba stubs compilados de outra equipe.

## Fase de integração — roteiro do grupo Cliente

- Receba do professor os dois Servidores REST e os dois Servidores gRPC sorteados para sua equipe.
- Registre código da equipe parceira, host/IP e porta ou base URL.
- Antes de executar a bateria gRPC, chame Health para confirmar que está alcançando o Servidor correto.
- Configure REST\_BASE\_URL e execute R1–R5 contra o primeiro Servidor REST; repita contra o segundo.
- Configure GRPC\_TARGET e execute G1–G5 contra o primeiro Servidor gRPC; repita contra o segundo.
- Não ajuste seu código para a tecnologia específica do Servidor. Corrija apenas problemas reais do seu Cliente mantendo o contrato.
- Para cada falha, registre em que etapa ocorreu: conexão, request, serialização, contrato, regra de negócio ou interpretação da resposta.
- Solicite ao grupo Servidor os logs correlacionados pelo mesmo Request ID para anexar ou confrontar no relatório.

## Formato mínimo de log do Cliente

```
[2026-09-08T20:00:00] protocol=REST client=C01 server=S03 requestId=req-101 operation=GET_PRODUCT
target=http://10.0.0.23:8080 status=200 durationMs=41 result=PASS
[2026-09-08T20:02:10] protocol=GRPC client=C01 server=S04 requestId=req-204 operation=CalculateShipping
target=10.0.0.24:50051 status=OK durationMs=18 priceCents=1800 result=PASS
```

## Matriz de execução do Cliente

| Protocolo | Servidor parceiro | Teste | Resultado inicial | Resultado final | Request ID | Observação |
|-----------|-------------------|-------|-------------------|-----------------|------------|------------|
| REST      | _____             | R1–R5 | _____             | _____           | _____      | _____      |
| REST      | _____             | R1–R5 | _____             | _____           | _____      | _____      |
| gRPC      | _____             | G1–G5 | _____             | _____           | _____      | _____      |
| gRPC      | _____             | G1–G5 | _____             | _____           | _____      | _____      |

## Entrega final do grupo Cliente

21. Código-fonte do Cliente REST e do Cliente gRPC.
22. README com tecnologia utilizada, dependências e comandos exatos para configurar e executar contra qualquer Servidor.
23. Logs completos das integrações REST e gRPC.
24. Relatório técnico em PDF ou DOCX conforme orientação do professor.

### Estrutura mínima do relatório

- Identificação da equipe e tecnologias escolhidas.
- Breve descrição da arquitetura dos dois Clientes.
- Tabela com os 2 Servidores REST e os 2 Servidores gRPC sorteados.
- Resultado de R1–R5 para cada Servidor REST e G1–G5 para cada Servidor gRPC.
- Trechos de logs do Cliente e do Servidor que compartilhem o mesmo Request ID.
- Falhas observadas, causa provável/confirmada e correções realizadas.
- Comparação prática REST × gRPC a partir da experiência da equipe.
- Conclusão: o que tornou uma integração interoperável ou incompatível.

## Checklist antes do sorteio

- ☐ REST\_BASE\_URL é configurável.
- ☐ GRPC\_TARGET é configurável.
- ☐ R1–R5 passam contra um ambiente local de teste.
- ☐ G1–G5 passam contra um ambiente local de teste.
- ☐ Headers REST obrigatórios são enviados.
- ☐ Metadata gRPC obrigatória é enviada.
- ☐ Request IDs são únicos e aparecem nos logs.
- ☐ O programa mostra claramente PASS/FAIL.
- ☐ Nenhum campo, path, enum ou regra foi inventado fora do contrato.
- ☐ README contém comando de execução reproduzível.

### Critério de sucesso

Seu Cliente não precisa conhecer como o Servidor foi construído. Se ele consegue conversar corretamente com implementações independentes apenas porque todos obedecem ao mesmo contrato, o objetivo principal da atividade foi atingido.